

Containers

Zdenko Pavlik

Scheidt&Bachmann Slovakia s.r.o.

Containers

... done with support of



Figure 1: scheidt&bachmann

Agenda

- ▶ Lessons learned from previous lesson (Rule of 5)
- ▶ Intro into containers
- ▶ Establishing containers for demonstration

Lessons learned from previous lesson (Rule of 5)

Class has several function, that are used when we alter lifecycle of class (either by new creation, assigning of copying from already existing classes). Each of them can be default, user defined or deleted.

Lessons learned from previous lesson (Rule of 5)

constructor

`NamedClass()`

Called when class is created. Can be default or parametrized.

`NamedClass newClass;`

copy constructor

`NamedClass(const NamedClass& other) // copy constructor`

Called when class is constructed from instance of another class (of the same type).

Member data are **copied**.

`NamedClass newClass = std::copy(otherClass);`

move constructor

`NamedClass(NamedClass&& other) noexcept // move constructor`

Called when class is constructed from instance of another class (of the same type).

Member data are **moved** and instance being moved from is not safe to use anymore!

Benefit is reuse of resources and in general it is faster than copying.

`NamedClass newClass = std::move(otherClass);`

Lessons learned from previous lesson (Rule of 5)

copy assignment operator

```
NamedClass& operator=(const NamedClass& other) // copy assignment
```

Called when content of the class is overridden by other class using **copy**.

```
newClass = std::copy(otherClass);
```

move assignment operator

```
NamedClass& operator=(NamedClass&& other) noexcept // move assignment
```

Called when content of the class is overridden by other class using **move**.

```
newClass = std::move(otherClass);
```

Lessons learned from previous lesson (Rule of 5)

Examples

```
NamedClass var1{"var1"};
```

```
NamedClass var2{var1};           // Calling copy constructor
```

```
NamedClass var3 = std::move(var1); // Calling move constructor
```

```
var1 = var2;                     // "Calling copy assignment"
```

```
var3 = std::move(var1);          // "Calling move assignment"
```

Intro into containers

Containers are pre-prepared storage for our data. There is no unique “best for all scenarios” container, each container is optimized for different purpose. Containers designed to store large amount of data in most effective way.

- ▶ generic (templated to any type, evaluated during compilation)
 - ▶ `std::list<int>`
 - ▶ `std::list<std::string>`
- ▶ safe to use, tested by tons of users
- ▶ heavily optimized

Benefits of using std containers

- ▶ `for (auto i : items){}`
- ▶ `container.find_if()`
- ▶ safety bounds checking (`std::vector[10]` on 5 vector of 5 elements -> not accessing random memory)
 - ▶ `container.at(10)`
- ▶ algorithms (`std::sort`)

Choosing containers for demonstration

I decide to choose three similar containers. All of them provide (almost) same functions, but performance differs a lot.

- ▶ `std::list`
- ▶ `std::vector`
- ▶ `std::deque`

Common characteristics

- ▶ `.emplace_back()` - optimized way to add at the end
- ▶ `.insert(position)` - insert element on given position (i.e. after 5th element)
- ▶ `.at(position)` - random access

Container brief overview

Organization in memory

Organization of **vector**, list, deque (and map, array)

`std::vector`



Figure 2: Vector - organization in memory

Adding element

Adding element to **vector**, list, deque (and map, array)



Figure 3: Vector - addition

Organization in memory

Organization of vector, **list**, deque (and map, array)

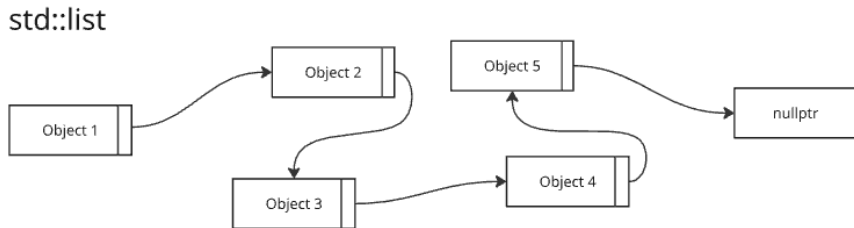


Figure 4: List - organization in memory

Adding element

Adding element to vector, **list**, deque (and map, array)

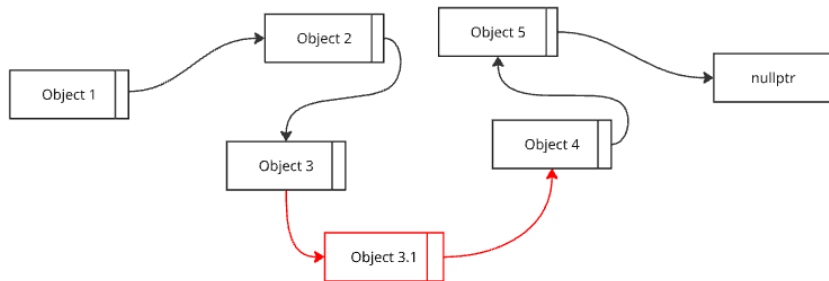


Figure 5: List - addition

Organization in memory

Organization of vector, list, **deque** (and map, array)

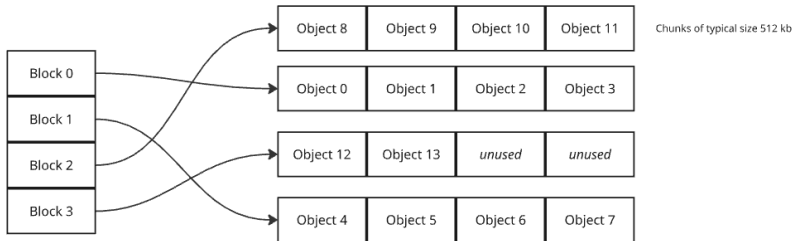


Figure 6: Deque - organization in memory

Adding element

Adding element to vector, list, **deque** (and map, array)

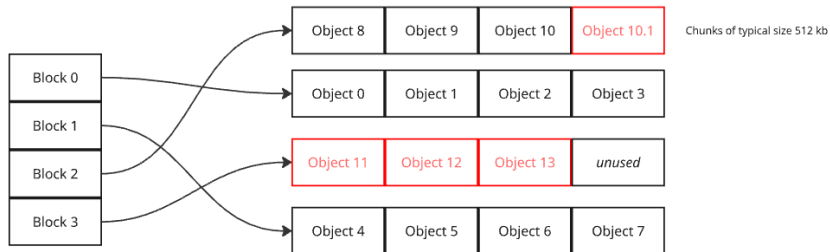


Figure 7: Deque - addition

Organization in memory

Organization of vector, list, deque (and **map**, array)

`std::map`

Other data #1	
id1	Object 1
id2	Object 2
id4	Object 3
id7	Object 4
Other data #1	

Figure 8: Map - organization in memory

Adding element

Adding element to vector, list, deque (and **map**, array)

Other data #1	
id1	Object 1
id2	Object 2
id3	Object 3.1
id4	Object 3
id7	Object 4
Other data #1	

Figure 9: Map - addition

Disclaimer map (binary tree (balanced))

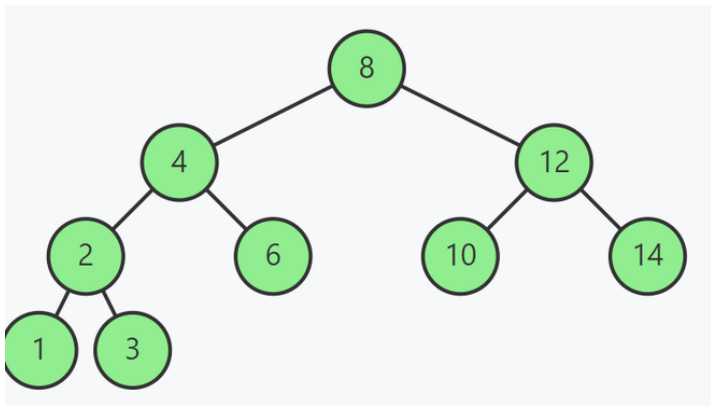


Figure 10: Map, balanced tree - disclaimer

[More here - Red black tree](#)

Organization in memory

Organization of vector, list, deque (and map, **array**)

`std::array`

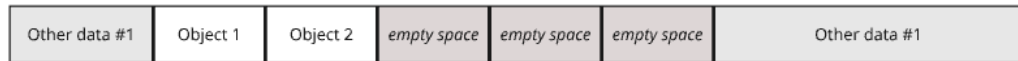


Figure 11: Array - organization in memory

Adding element

Adding element to vector, list, deque (and map, **array**)



Figure 12: Array - addition

Algorithm complexity (BigO notation)

	Access by index	Search	Insertion	Deletion
<code>std::array</code>	$O(1)$	$O(N)$	N/A(fixed size)	N/A(fixed size)
<code>std::vector</code>	$O(1)$	$O(N)$	$O(N)$ or $O(1)$	$O(N)$ or $O(1)$
<code>std::list</code>	$O(N)$	$O(N)$	$O(1)$	$O(1)$
<code>std::map</code>	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$
<code>std::deque</code>	$O(1)$	$O(N)$	$O(N)$ or $O(1)$	$O(N)$ or $O(1)$

Demonstration

On `std::list`

On `std::vector`

On `std::deque`

Lessons learned

Think before you choose a container, because it can have tremendous impact on performance. As well you can cause unintended memory deallocation/allocation.

- ▶ Do I need to add elements frequently?
- ▶ Do I need to remove elements frequently?
- ▶ Do I need to add elements in the middle ?
- ▶ Do I need to have continuous memory?
- ▶ etc..

links

Where to use particular container